

Plano de Ação de Auditoria Consolidado: ZebTrack-AI

De: Arquiteto de Software Sênior / Engenheiro de QA **Assunto:** Plano de implementação unificado com base em todas as auditorias (Nexus, Agente 1, Agente 2) **Prioridade:** 1º Estabilidade, 2º Manutenibilidade, 3º Consistência.

Este documento sintetiza as descobertas de três auditorias independentes e estabelece um plano de ação priorizado para corrigir riscos de estabilidade e débitos técnicos.

Fase 1: Estabilidade Imediata (Correções Críticas P0)

(Foco: Impedir que a aplicação trave, vaze recursos, corrompa dados ou entre em deadlock)

1.1. [CRÍTICO] Corrigir Vazamento de Threads e Race Condition no LiveCameraService

- **Referência:** Auditoria 1 (`AUDIT_REPORT.md`), Auditoria 2 (`RELATORIO_AUDITORIA_ZEBRATRACK.md`)
- **Problema:** O `LiveCameraService` inicia threads (`LiveCameraCaptureThread` , `LiveCameraProcessingThread`) que não são finalizadas (`stop_session` não é chamado), causando um vazamento massivo de threads. A *causa raiz* é o estado compartilhado (buffers de frames, flags) sendo acessado sem `threading.Lock` , criando Condições de Corrida (*Race Conditions*).
- **Imprevisto:** A aplicação irá travar por exaustão de recursos (threads) após algumas sessões de análise ao vivo.
- **Ação:**
 1. Implementar `threading.Lock` no `LiveCameraService` para proteger o acesso a todos os estados compartilhados.
 2. Refatorar o `LiveCameraService` para implementar o padrão *Context Manager* (`__enter__` e `__exit__`), garantindo que `stop_session()` seja chamado de forma robusta dentro do `__exit__` ou em um bloco `finally` .
 3. **Validação:** Atualizar `test_live_camera_analysis_integration.py` para usar `with LiveCameraService(...) as service:` e verificar (ex: via `threading.active_count()`) que as threads foram de fato encerradas.

1.2. [CRÍTICO] Corrigir Risco de Divisão por Zero na Validação

- **Referência:** Auditoria 2 (`RELATORIO_AUDITORIA_ZEBRATRACK.md`)
- **Problema:** Lógica de validação (provavelmente em calibração ou configuração de ROI) que não verifica se um divisor (ex: `pixels_por_cm`) é zero antes de uma operação de divisão.
- **Imprevisto:** Crash imediato da aplicação com `ZeroDivisionError` se o usuário fornecer entradas inválidas (ex: pontos colineares na calibração).
- **Ação:** Adicionar verificações de robustez (`if divisor is not None and divisor != 0:`) antes de todas as operações de divisão em módulos de calibração (`calibration.py`) e validação de configuração.

1.3. [CRÍTICO] Corrigir Risco de Deadlock no StateManager

- **Referência:** Minha Auditoria (Nexus) (`auditoria_zebtrack.md`)
- **Problema:** O `timeout` de segurança para *observers* no `StateManager` usa `signal.SIGALRM`, que **não funciona no Windows nem em threads** (apenas na thread principal do Unix).
- **Imprevisto:** Um *observer* (ouvinte) que entrar em deadlock ou loop infinito irá **congelar a aplicação inteira**, e a proteção de `timeout` falhará silenciosamente.
- **Ação:** Remover imediatamente a implementação de `timeout` baseada em `signal.SIGALRM` de `src/zebtrack/core/state_manager.py` (método `timeout` e seus usos em `_notify_observers`). É preferível aceitar o risco de um *observer* lento a ter uma falsa sensação de segurança que falha em produção.

1.4. [CRÍTICO] Eliminar Mascaramento de Erros no MainViewModel

- **Referência:** Minha Auditoria (Nexus) (`auditoria_zebtrack.md`)
- **Problema:** Uso generalizado de `except Exception:` no `MainViewModel`.
- **Imprevisto:** Erros de programação (ex: `TypeError`, `AttributeError`) são silenciados e logados da mesma forma que erros operacionais (ex: `FileNotFoundError`), tornando a depuração impossível e permitindo que a aplicação continue em estado corrompido.
- **Ação:** Substituir todos os blocos `except Exception:` por exceções específicas da hierarquia em `src/zebtrack/core/exceptions.py`. Deixar que `TypeError`, `AttributeError`, etc., falhem ruidosamente (Fail-Fast).

Fase 2: Refatoração Arquitetural (Dívida de Longo Prazo P1)

(Foco: Corrigir o design para impedir que novos bugs surjam e facilitar testes)

2.1. [ALTO] Desacoplar View do ViewModel (Inversão de Controle)

- **Referência:** Minha Auditoria (Nexus)
- **Problema:** O `MainViewModel` (MVM) instancia a `ApplicationGUI` (`self.view = ...`), criando um acoplamento rígido que impede testes de unidade do MVM.
- **Ação:** Inverter a dependência.
 1. Remover `self.view = ApplicationGUI(...)` do `MVM.__init__`.
 2. No ponto de entrada (`__main__.py`), instanciar `state_manager` e `main_view_model` primeiro.
 3. Instanciar `ApplicationGUI(root, main_view_model, state_manager)`.
- **Justificativa:** Esta é a ação **mais importante** para a manutenibilidade a longo prazo, pois permite que o MVM seja testado em isolamento (mockando o `state_manager` e o `event_bus`).

2.2. [ALTO] Corrigir Testes Ineficazes (Sintoma do Acoplamento)

- **Referência:** Auditoria 1 (`AUDIT_REPORT.md`)
- **Problema:** Os testes de unidade (ex: `test_live_camera_service_threading.py`) fazem *mock* (simulam) dos métodos de limpeza (`stop_session`).
- **Imprevisto:** O teste passa (fica "verde"), mas a implementação real vaza recursos (o bug P0 da Fase 1). O teste valida uma fantasia, não a realidade.

- **Ação:** Após o desacoplamento (2.1), refatorar os testes para *não* fazer mock dos métodos de limpeza. Os testes devem validar o ciclo de vida completo (setup, execute, teardown).

2.3. [ALTO] Desmembrar o "God Object" (MainViewModel)

- **Referência:** Todas as Auditorias
- **Problema:** O `MainViewModel` (MVM) é um "God Object" com >2000 linhas que atua como um roteador central para toda a lógica de negócios, resultando em complexidade ciclomática extrema (ex: `process_pending_project_videos`, C901).
- **Ação:**
 1. Mudar o padrão de despacho de eventos. Em vez do MVM rotear tudo via `_EVENT_METHOD_MAPPING`, os Orquestradores (ex: `RecordingSessionOrchestrator`) devem se inscrever *diretamente* no `EventBus` para os eventos que lhes interessam (ex: `Events.RECORDING_START`).
 2. Refatorar `process_pending_project_videos` em métodos menores e privados, conforme sugerido pela Auditoria 1.
 3. Remover lógicas de *fallback* (ex: `_start_live_camera_analysis_direct`) que duplicam a lógica dos orquestradores (Minha Auditoria).

2.4. [MÉDIO] Simplificar Lógica de Estado (Global vs. Projeto)

- **Referência:** Minha Auditoria (Nexus)
- **Problema:** O MVM mantém um estado `_global_model_defaults` que é restaurado ao fechar um projeto, criando uma lógica de estado confusa para o usuário.
- **Imprevisto:** O usuário altera uma configuração global, abre um projeto (que sobrescreve a configuração) e, ao fechar, a configuração global é "magicamente" restaurada, dando a impressão de que a alteração foi perdida.
- **Ação:** Remover a lógica de `_global_model_defaults` e `_restore_global_model_defaults`. O `StateManager` deve ser a única fonte da verdade, carregando as configurações do usuário salvas em disco (`config.yaml`) ao fechar um projeto.

Fase 3: Higienização e Consistência (P2)

(Foco: Melhorar a qualidade de código e a legibilidade da arquitetura)

3.1. [MÉDIO] Corrigir Vazamentos de Recursos de Arquivos e Captura

- **Referência:** Auditoria 1 (`AUDIT_REPORT.md`), Auditoria 2 (`RELATORIO_AUDITORIA_ZEBRATRACK.md`)
- **Problema:** Múltiplos `ResourceWarning: unclosed file` e potenciais vazamentos de `cv2.VideoCapture`.
- **Ação:** Refatorar todas as operações de I/O de arquivo para usar o padrão `with open(...)` as `f`: e garantir que `cv2.VideoCapture` seja sempre liberado (`cap.release()`) em blocos `finally` ou *context managers*.

3.2. [MÉDIO] Consolidar Nomenclatura Arquitetural

- **Referência:** Minha Auditoria (Nexus)

- **Problema:** Confusão entre `core.*_coordinator` , `coordinators.*` e `orchestrators.*` .
- **Ação:** Padronizar em um único nome (ex: "Orchestrators") e mover todas as classes de alto nível para `src/zebtrack/orchestrators/` . Renomear ou fundir os pacotes duplicados.

3.3. [MÉDIO] Corrigir Dívida Técnica de Aliases no StateManager

- **Referência:** Minha Auditoria (Nexus)
- **Problema:** O `StateManager` possui `_normalize_state_kwargs` para mapear chaves legadas (ex: `is_detector_initialized`) para novas (`detector_initialized`).
- **Ação:** Fazer uma busca global por todas as chaves legadas e refatorá-las para usar as chaves canônicas. Remover `_normalize_state_kwargs` após a conclusão.

3.4. [BAIXO] Executar Linter

- **Referência:** Auditoria 1 (`AUDIT_REPORT.md`), Auditoria 2 (Ruff)
- **Ação:** Executar `poetry run ruff check . --fix` para corrigir automaticamente a maioria das 130+ violações de estilo.
- **Justificativa:** Baixo esforço, alto ganho em consistência de código.